



Design Document

Version 1.0

Luke Smith

Completed 05/25/2026

Ryan Hammer: Stakeholder

Revision History

Version	Date	Author	Description
0.1	May 17, 2026	Luke Smith	Created design document, added table of contents with all sections, and anticipated structure
0.2	May 18, 2026	Luke Smith	Added background and stakeholders, UI mockups, and a link to the full generated mockup
0.3	May 22, 2026	Luke Smith	Worked on a large proportion of the document: filled out the scope, a large portion of the user stories, non-functional requirements, high-level overview, UI mockup descriptions, security considerations, and made good progress on the appendix, testing, and launch and deployment plan
0.4	May 23, 2026	Luke Smith	Started acceptance criteria, finished design issues, started sequence diagrams, finished user stories, testing, appendix, launch and deployment plan, and state + activity diagram
0.5	May 24, 2026	Luke Smith	Worked on acceptance criteria and ensured all sections are complete, except diagrams
1.0	May 25, 2026	Luke Smith	Completed sequence diagrams, class diagram, and acceptance criteria. Design document reviewed and uploaded to the repository

Table of Contents

● Purpose	4
○ Project Details	
○ Functional Requirements	
○ Non-Functional Requirements	
● Design Outline	12
○ High-Level Overview	
● Design Issues	15
○ Functional Issues	
○ Non-Functional Issues	
● Design Details	19
○ Class Design	
○ API Endpoints	
○ Sequence Diagrams	
○ State / Activity Diagrams	
○ UI Mockups	
● Security Considerations	28
● Testing Strategy	29
● Launch & Deployment Plan	31
● Appendix	32

Purpose

Project Background

This project is a hub for the BlueBlooded podcast and a consolidation of all Ryan Hammer content. It will include the podcast and short-form content, and discussion boards for all relevant topics to the users. Along with that, statistics from *ESPN* or other college basketball statistic websites are going to be used to keep up with relevant statistics that Ryan Hammer uses consistently in his content.

Problem Statement

The college basketball world has expanded so much as analytics, statistics, and viewing experiences have become much more accessible for the average fan. Content creators such as Ryan Hammer have created comprehensive platforms that condense this information into easily accessible formats. However, creators post on TikTok, Instagram, and other social media outlets, and do not have a complete consolidation of all of the information they provide and the statistics they use. The goal of this app is to consolidate all of that information, creating one application where you can see commonly used statistics, analysis of current college basketball landscapes, and other content from creators.

Goals & Objectives

1. Create a consolidated platform that contains college basketball news, analysis, and statistics.
2. Allow content for creators like Ryan Hammer to upload material such as podcasts or short-form videos.
3. Have an easily accessible platform, where users can understand information and know where to go for it.
4. Collaborate with established statistical websites and allow creators like

Ryan Hammer to see trends more clearly

5. Develop a successful multi-platform prototype for a July launch as a solo developer

Scope

A college basketball application for Ryan Hammer, with a minor amount of NBA Draft content. Scores, rosters, and overall team information that is not statistical will not be covered.

Stakeholders & Users

Primary Stakeholder: Ryan Hammer

End Users: CBB Fans and the audience of BlueBlooded

Development Team: Luke Smith

Functional Requirements

Authentication & User Management

User Story	Priority	Effort	Acceptance Criteria
As a user, I would like to sign up for an account to participate in discussion posts.	High	S	Creating an account works; invalid emails are not allowed; invalid passwords are not valid; duplicate accounts are not allowed
As a user, I would like to be able to log in to a previously created account	High	S	Logging into a non-existent account is not allowed. You can log into a created account, but you can only log into your account with your password
As a user, I would like to reset my password	High	S	Can't reset with the same password, can sign in with a new password, can not sign in with the old password
As a user, I would like to have a bio that says a little about myself	Low	S	Bio is displayable, editable, and can be seen by other users
As a user, I would like to be able to attach myself to a certain school that I root for	Low	S	All valid schools are accessible, a chosen school is displayed to other users, and a chosen school persists over multiple restarts
As a user, I would like to be able to change details of my profile, such as school, username, bio, etc.	Medium	S	School is changeable, bio is changeable, username is changeable, and changes are displayed to the user and other users
As a user, I would like to create an avatar that users can see when checking my profile	Low	M	Avatars can be created, seen by other users, and are customizable to differentiate between users

CBB Statistics Hub

User Story	Priority	Effort	Acceptance Criteria
As a user, I would like to see relevant statistics from the current basketball season	High	M	Statistics are displayed for the current college basketball season, are accurate, and are updated consistently throughout the day

As a user, I would like to see an actively-updating Trapezoid of Excellence	High	L	Trapezoid of excellence appears on the stats page, teams appear on the graph, and teams that should be in the trapezoid are
As a user, I would like to see an actively-updating list of potential national champions via statistics	Medium	L	Teams that are eligible for a national championship, relevant statistics are shown, and eliminated teams are not displayed
As a user, I would like to see the best and worst teams in certain relevant statistical categories	Medium	M	The best teams for a statistical category are shown, the worst teams are shown, and best to worst is ordered properly

Podcast / Content Player

User Story	Priority	Effort	Acceptance Criteria
As a user, I would like to be able to watch the most recent episode of the BlueBlooded podcast	High	M	Episodes of BlueBlooded are available to be seen, and the most recent episode is accurately displayed at the top
As a user, I would like to view the uploaded content from Ryan Hammer	High	S	Content from Ryan Hammer appears, can be viewed, and can be replayed
As a user, I would like to be able to read a description of the podcast episode before I watch it	Medium	S	Podcast episodes appear properly, a description box is rendered, and an accurate description for the episode is given
As a user, I would like to have sections in the podcast episode so I could skip to what I wanted to listen to	Low	L	Users see sections integrated in the podcast episode, are able to select a specific section to skip to, and can return to a previous section properly
As a user, I would like to be able to rewatch older episodes of the BlueBlooded podcast	High	S	Users see older episodes of BlueBlooded, can watch them with no issue, and they are sorted from newest to oldest
As a user, I would like to be able to search for a specific episode of the BlueBlooded podcast	Medium	M	Users are able to search on the podcast page, the proper episodes show up for a search, and nothing shows up for invalid searches

Community Discussion Board

User Story	Priority	Effort	Acceptance Criteria
As a user, I would like to create a discussion post about a given topic.	High	M	Discussion post can be created, the discussion post is displayed to the user, and the discussion post is displayed to other users
As a user, I would like to be able to see discussion posts created by other users	High	S	Users can see discussion posts and other users' posts. Posts that do not exist should not appear
As a user, I would like to reply to a discussion post via a comment	Medium	S	Users have the option to reply to a discussion post, the text input is correctly displayed, and other users can see the reply
As a user, I would like to be able to delete a reply or comment	Low	S	Users can delete a reply/comment, and other users can not see the deleted reply/comment
As a user, I would like to be able to like comments or replies that I find worthy	High	S	Users can like a comment, like a reply, and unlike either whenever they see fit
As a user, I would like to be able to search or filter threads by category	Medium	M	Users are able to filter or search in the discussion post tab, given filters accurately filter to the desired discussion posts, and filters that have no corresponding topics should not display anything

Admin / Content Management

User Story	Priority	Effort	Acceptance Criteria
As an admin, I would like to post content for other users to see.	High	M	Admins can post content, the proper content is posted to the app, and the users can see the posted content
As an admin, I would like to delete posts that I do not want to be seen by others at the current moment.	High	S	Admins have the option to delete posts, posts are properly moved from the UI, and users can not search for the given content anymore
As an admin, I would want a special tag denoting that I am an admin, and that it is my official account	Medium	S	An admin tag appears for only admin individuals; other users can see the admin tag, and those with the admin tag are the only ones with special privileges
As an admin, I would like to be	Medium	M	Admins are able to ban any user; users that

able to ban people who are harming the structure of the app			are banned can not access the app on their given account, and admins can not ban other admins.
As an admin, I would like to be able to delete discussion posts or comments that are inappropriate for the app.	High	S	All discussion posts can be deleted by admins, all comments can be deleted by admins, and deleted comments/posts disappear for users

Non-Functional Requirements

Performance

Statistics are expected to be displayed accurately in less than or equal to 2 seconds, and transitions between pages should take no more than one second, as one second would be assumed for pages with a large amount of digital content, such as graphs or podcasts.

Scalability

The backend should be able to support 10,000 users concurrently without a decrease in the quality of the product. Along with that, FastAPI and React Native should allow for scaling without a change of tech stack.

Security

Plain text credentials will be encrypted for storage, and authentication as an admin will be required for posting episodes or other media onto the content, excluding discussion boards. All data will be transmitted over HTTPS.

Availability & Reliability

Users should be able to get what they want out of the BlueBlooded app at all times. Not assuming scheduled maintenance, users should be able to navigate to the content they want to consume efficiently and without delay, within 10 seconds.

Platform & Compatibility

We are building a cross-platform app, so users should be able to use any mobile device, and all features should work concurrently across all platforms.

Maintainability

Code structure will consist of two separate directories for frontend and backend

code, components created separately with React Native, API endpoints, and a clear definition of files to understand where code lies. Maintenance with this setup will be efficient and thorough.

Design Outline

High-Level Overview

1. Client

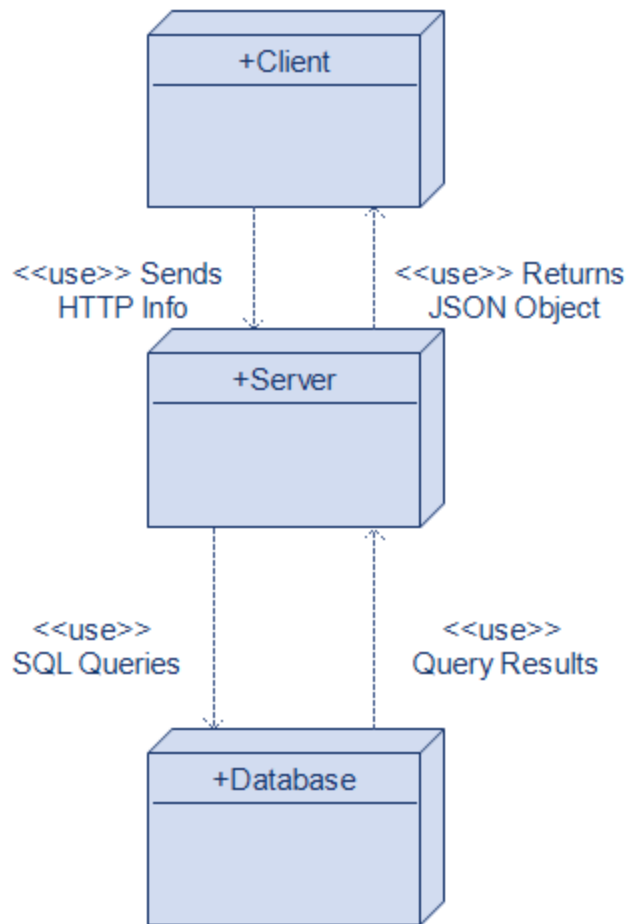
- a. Serves as the main interface of the user
- b. Allows users to interact with Ryan Hammer's content, view CBB statistics and models, and view news about the sport as a whole
- c. Sends inputted information as HTTP requests to the server and receives responses in JSON format
- d. Parses the data received from the server and displays it to a user

2. Server

- a. Serves as the central coordinator between the client and the database.
- b. Receives and processes HTTP requests from the client.
- c. Validates requests and queries the database to retrieve or update data.
- d. If data is requested, the server returns the required information to the user.
- e. Pulls profile information from the database
- f. Pulls from specifically queried stats APIs to provide accurate statistics to users

3. Database

- a. PostgreSQL relational database management system stores and maintains CBB statistics (Trapezoid of Excellence, potential national champions) for user access and Ryan Hammer manipulation into meaningful content.
- b. Responds to queries from the server, updates statistical or profile information as needed



Tech Stack

Backend: FastAPI (Python)

Frontend: React Native + Expo

Database: PostgreSQL

External Integrations

- KenPom API
- ESPN Public API

Deployment

- Backend hosted on Render
- Database hosted on Supabase
- Expo EAS builds the app

- App Store / Google Play distributes the app

Design Issues

Functional Issues

1. How often will stats be updated?
 - a. Every Minute
 - b. Every Hour
 - c. Every Day

Choice: Option B

Reasoning: College basketball statistics do not change by the minute, and cost and rate limiting would spike due to consistently calling an API. On days such as weekends, when there are a ton of games, daily updates will not represent the changes throughout the day. Hourly updates are the perfect middle ground between less rate limiting and consistently accurate statistics.

2. What is the strategy for caching?
 - a. No caching
 - b. Cache in the stats_cache table in Supabase with a TTL tied to the update frequency
 - c. Cache on the FastAPI server in memory

Choice: Option B

Reasoning: In-memory caching would be lost on server resets and could not scale for multiple servers. Along with that, fetching from the API each time, if we did not cache, would exhaust rate limits and violate our pre-determined performance rules. Employing a cache in Supabase with an

expires_at column helps the cache to persist, and fit the stats_cache table so everything is still queryable.

3. How should the podcast be hosted?
 - a. RSS feed from existing podcast host
 - b. Supabase Storage

Choice: Choice A

Reasoning: Storing full audio or video files in Supabase Storage would get expensive fast and introduces unnecessary complexity around streaming, buffering, and bandwidth. Ryan has a podcast host. Pulling from an RSS feed means the app just displays metadata and links out or embeds the stream, which is lighter, cheaper, and more maintainable. This also means Ryan can manage his episodes through tools he already knows.

4. What features will unverified users have access to?
 - a. All features
 - b. All features excluding discussion
 - c. Just the content viewer
 - d. No features

Choice: Choice B

Reasoning: Users should not be able to participate in discussion, as they will be unable to be located, identified, and banned for inappropriate behavior. However, statistics and the podcast should be open to the public, as this is open source content by Ryan and other places. Therefore, the account should only add discussion, and everything else should be freely accessible.

Non-Functional Issues

1. How will we handle the App Store review window given the July 14 submission target?
 - a. Submit on July 14 and hope the review is completed before the target launch date
 - b. Submit by July 7 to build in a one-week buffer for rejection and resubmission
 - c. Submit a bare-minimum build earlier in June to get one review cycle

Choice: Choice B

Reasoning: July 7th is the proper middle ground between the two extremes. The July 14th submission would leave no room for error on a rejection for Apple's one to three-day review window, and a bare-minimum June build risks rejection due to it not reflecting the final product. A July 7th submission allows for rejection and resubmission, while not submitting a product that is not polished enough to release.

2. How will we ensure feature parity between iOS and Android in the Expo build pipeline?
 - a. Use Expo EAS to build both platforms and test them simultaneously
 - b. Use a physical device for one platform and a simulator for the other
 - c. Build and test on iOS, address Android issues after launch

Choice: Choice A

Reasoning: It is a large risk to just test on iOS, as it would take a long time to solve Android-specific bugs after launch. Expo EAS allows for easy simultaneous testing and allows us to find bugs in the code at a much earlier

stage, which will cost less to fix. React Native handles most parity, and where iOS and Android diverge can be easily handled by the developer.

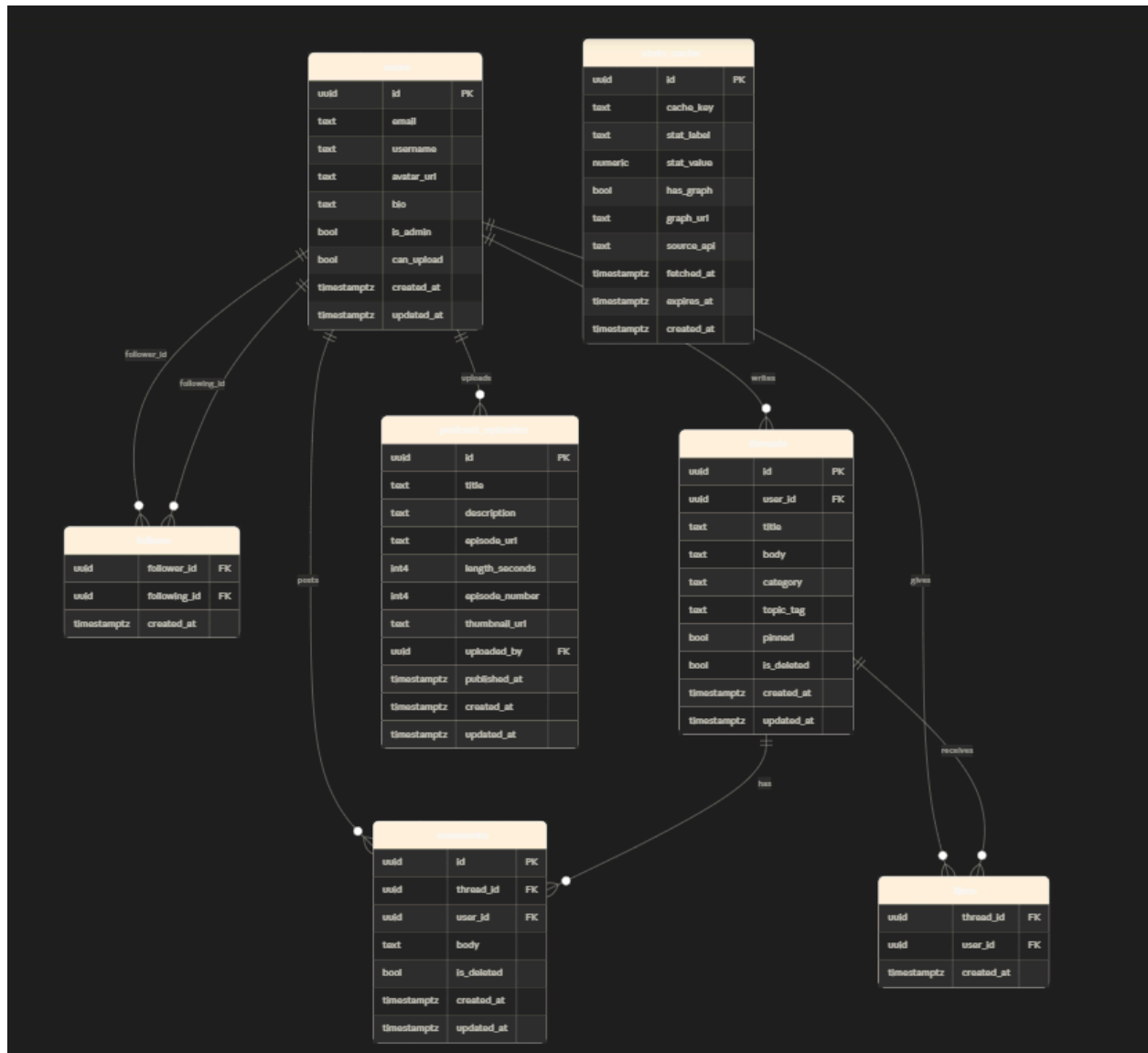
3. How will we manage CBB API rate limits to avoid service degradation?
 - a. Implement a request queue on the FastAPI backend
 - b. Fetch directly from the API on every user request with no limits
 - c. Rely on the hourly caching strategy to reduce API calls, and monitor usage

Choice: Choice C

Reasoning: The hourly caching decision in functional issue #1 already solves most of this problem; if stats are cached, the external API is only called once per hour, regardless of how many users are active. A request queue adds complexity that isn't warranted at this stage. Monitoring usage through the stats_cache fetched_at timestamps gives visibility into actual call frequency and makes it easy to tighten the TTL if rate limit issues emerge in production.

Design Details

Class Design



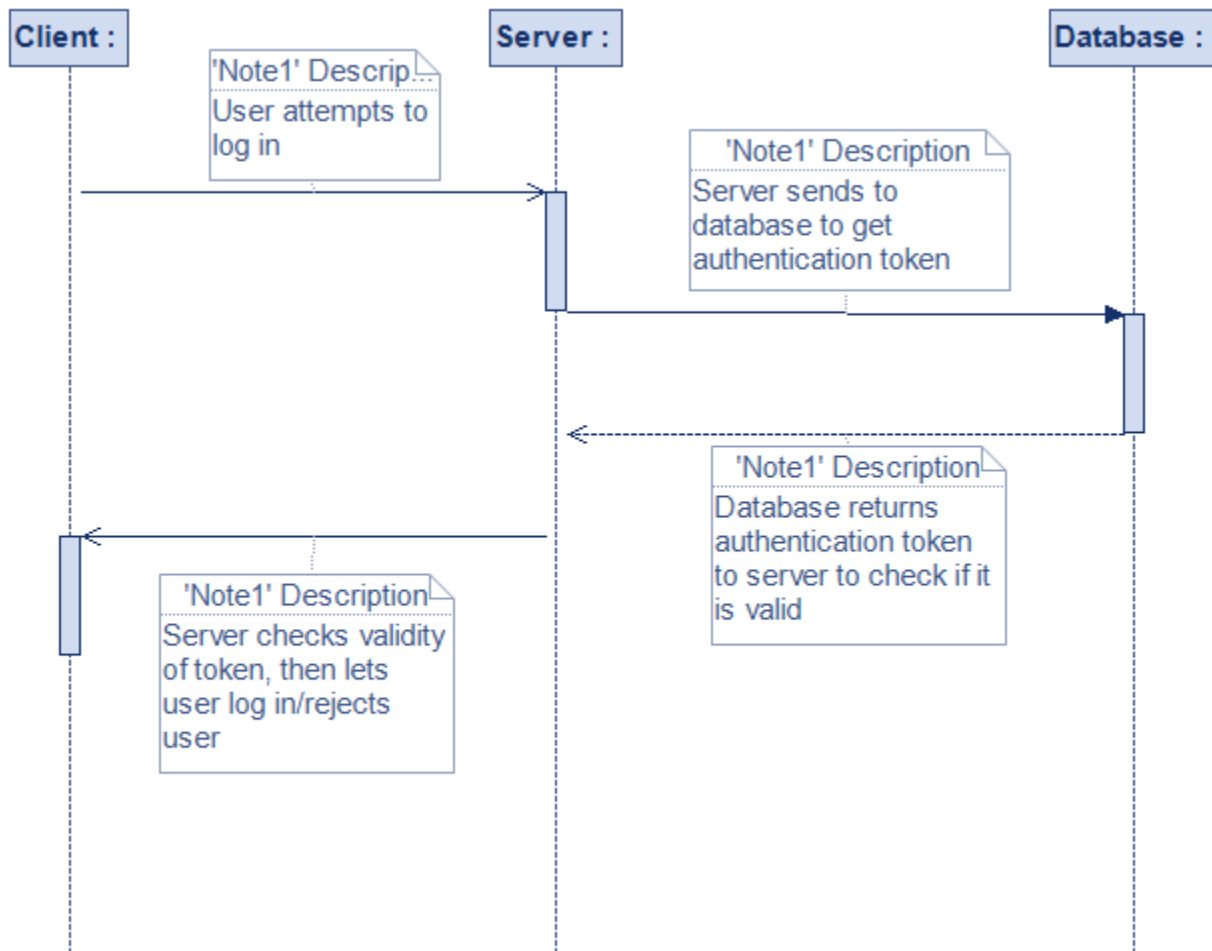
API Endpoints

Method	Endpoint	Purpose
POST	/auth/verify	Verify Supabase JWT and return user profile
GET	/users/{user_id}	Get a user's public profile
PATCH	/users/{user_id}	Update profile (bio, school, username, avatar)
GET	/stats/trapezoid	Get current Trapezoid of Excellence data
GET	/stats/champions	Get national champion candidate list
GET	/stats/categories	Get best/worst teams by statistical category
GET	/episodes	Get all episodes (paginated)
GET	/episodes/latest	Get the most recent episode
GET	/episodes/{episode_id}	Get a single episode with description
GET	/episodes/search?q=	Search episodes by title/description
POST	/episodes	Upload a new episode (admin only)
DELETE	/episodes/{episode_id}	Delete an episode (admin only)
GET	/threads	Get all threads, supports ?category= filter
GET	/threads/{thread_id}	Get a single thread
POST	/threads	Create a new thread
DELETE	/threads/{thread_id}	Delete a thread (own or admin)
GET	/threads/{thread_id}/comments	Get all comments on a thread

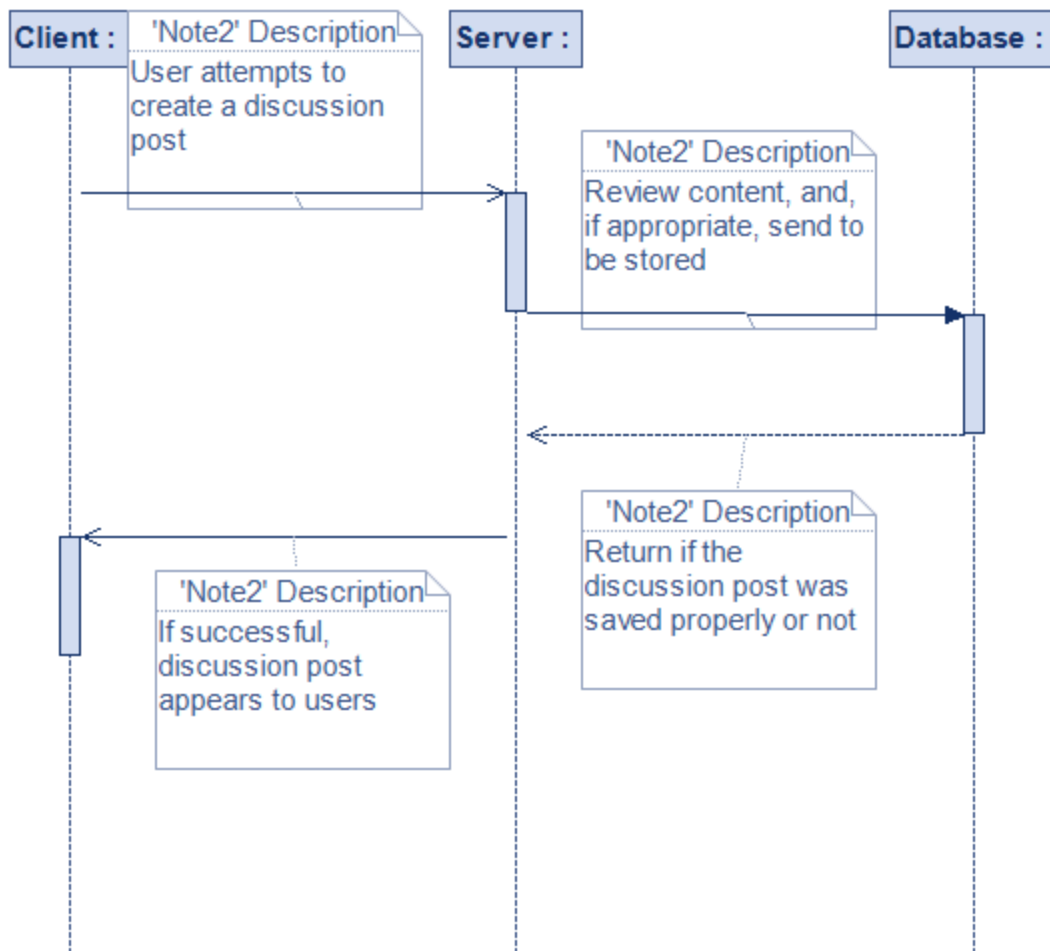
POST	/threads/{thread_id}/comments	Post a comment on a thread
DELETE	/comments/{comment_id}	Delete a comment (own or admin)
POST	/comments/{comment_id}/like	Like a comment
DELETE	/comments/{comment_id}/like	Unlike a comment
PATCH	/admin/users/{user_id}/ban	Ban a user

Sequence Diagrams

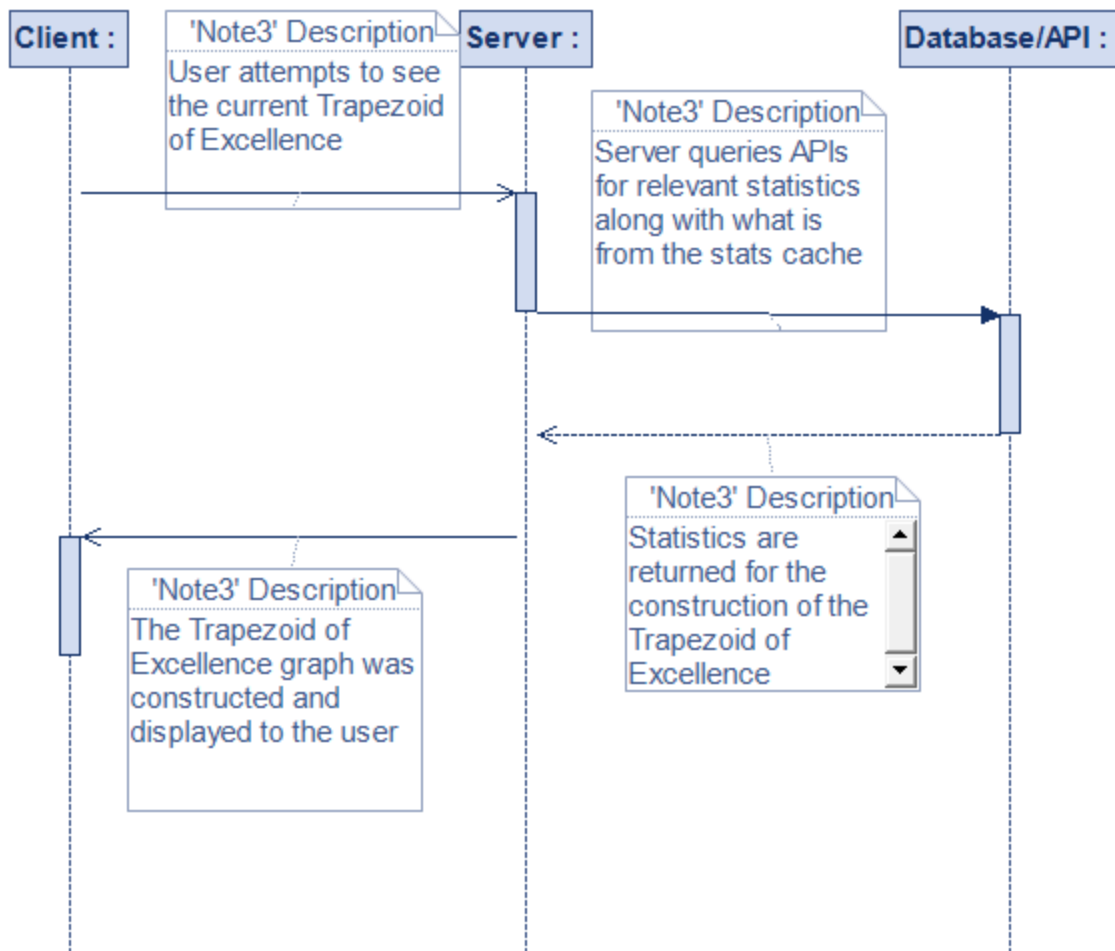
Logging in



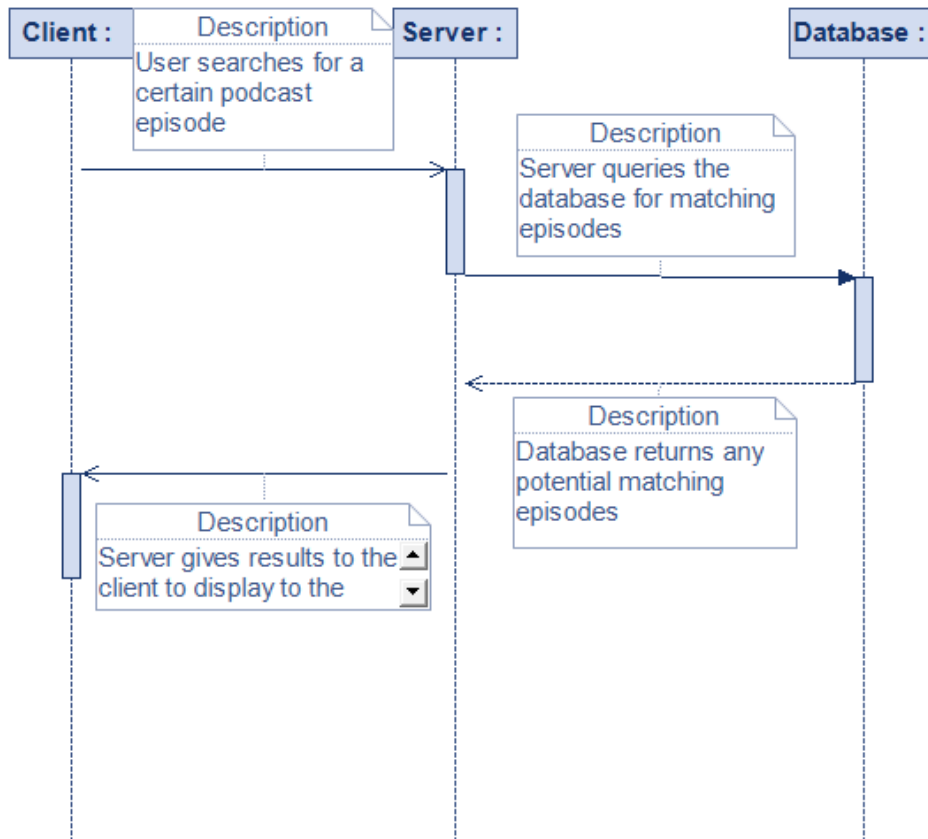
Creating a discussion post



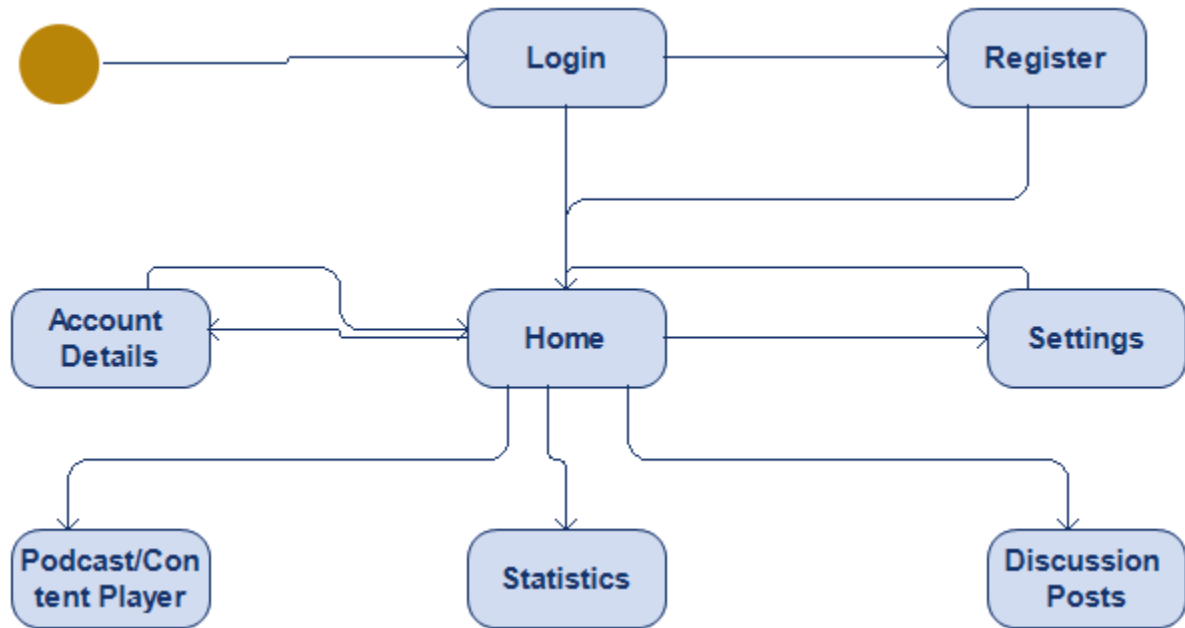
Viewing statistics



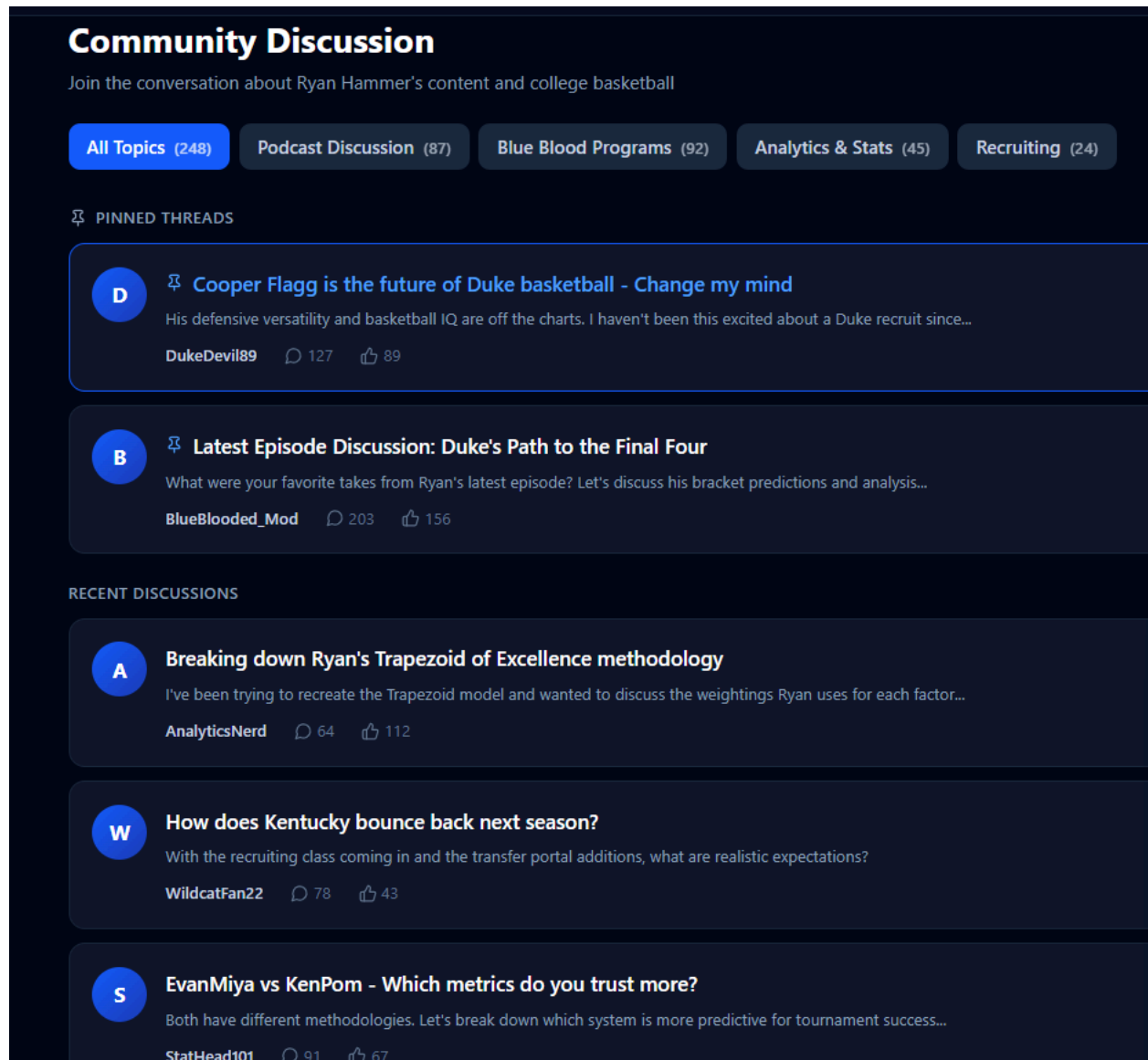
Searching episodes



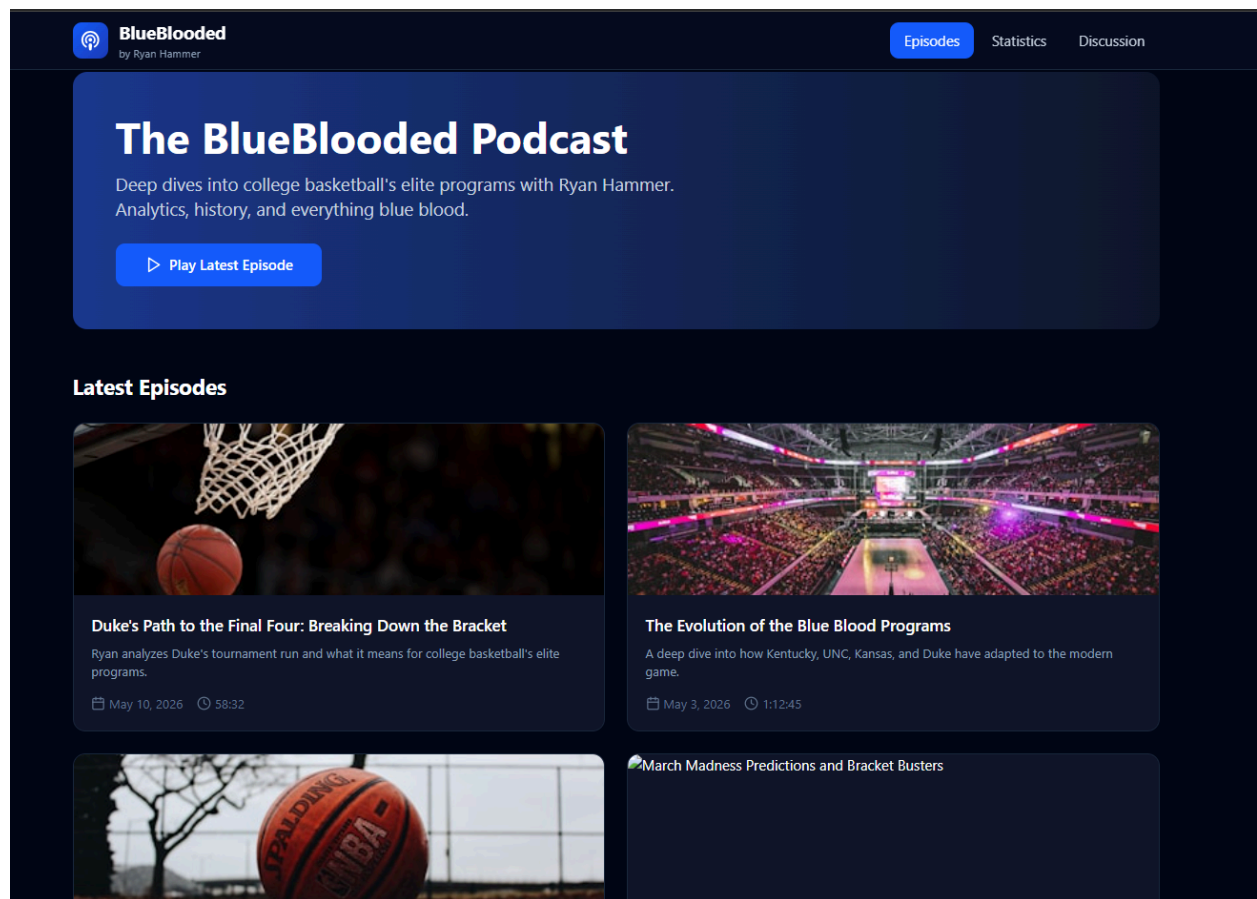
State / Activity Diagram



UI Mockups



A mock of the community feed, where users can make posts about anything and talk to other college basketball/Ryan Hammer fans about it



A mock of the podcast and social feed page for the app. A hub where you can see new episodes of BlueBlooded, short-form content from Ryan, and other important college basketball news.

Security Considerations

Authentication

Logins will be authenticated via Supabase auth and JWT. This is done by someone signing in on the expo app, Supabase taking those details, creating an access token, and FastAPI seeing if the access token is valid. If it is, the user will be logged in properly.

Authorization

Users who are authorized to upload media or delete media will be identified by a flag in their profile. There should be a small number of authorized users (3-5), so accounts will be identified for users who should be authorized, and the accounts will gain authorization privileges accordingly.

Data Privacy

All data meant to be kept private will be encrypted. This includes usernames, passwords, emails, or any other sensitive info that is input in the app. If there are in-app transactions, credit card details are not planned to be stored for user safety, in case there is a breach.

Testing Strategy

Unit Testing Plan

Frontend and Backend Unit Tests will be implemented throughout this project.

Frontend tests will consist of:

- Proper rendering of podcast episodes
- Rendering statistical components
- Validation for login/signup
- Screen navigation
- Discussion post creation forms

Backend tests will consist of:

- Admin/Authorization Checks
- Database Query Functions
- Statistics Formatting
- Calculation of Statistics Properly
- API request validation
- Supabase JWT verification logic

Integration Testing Plan

Tests will include:

- Expo app sends request to FastAPI successfully
- FastAPI verifies Supabase authentication tokens
- FastAPI reads and writes data from Supabase PostgreSQL
- Users can view and create discussion posts
- Admin users can upload, edit, and delete content
- Stats endpoints return accurate and properly formatted information

- Unauthorized users cannot access admin-only endpoints

User Acceptance Testing (with Ryan)

User acceptance testing will be completed with Ryan Hammer and a small group of target users.

Ryan will test whether:

- The app represents the BlueBlooded brand correctly
- Podcast/content pages are easy to navigate
- Statistics are displayed in a useful way
- Admin/content management tools are understandable
- The discussion board supports the type of community interaction he wants

Target users will test whether:

- They can create an account and log in
- They can find podcast episodes
- They can understand the statistics section
- They can create and reply to discussion posts
- The app feels smooth and intuitive on mobile devices

Feedback from UAT will be recorded and converted into backlog items before launch.

Launch & Deployment Plan

Tentative Plan

May 26: Complete design documentation, start planning implementation

May 29: Finish first sprint plan, begin first sprint

June 19: Finish first sprint, evaluate progress, check in with Ryan

June 22: Finish second sprint plan, begin second sprint

July 13: Finish second sprint, evaluate ability to be launched

July 14: Submit to App Store and Google Play for upload

App Store Submission Checklist

Before submission to the app store for usage, our application must:

1. Have proper authentication
2. Unauthorized users are rejected
3. Admin-only actions are handled properly
4. Main screens load quickly and efficiently
5. Database data is accurate
6. The app works on iOS and Android
7. No major crashes occur
8. Ryan is happy with the design and layout

Rollback Strategy

In the event of a broken deployment, the backend can be reverted to the previous build via Render's deploy history. Database migrations will be written to include a corresponding rollback migration. Expo EAS allows redeployment of any previous build.

Appendix

Glossary

CBB - College Basketball

Trapezoid of Excellence - A graph with a Trapezoid placed on it, showing the best teams in the country overall, and at adjusting to other teams' styles to win

Figma Link to the full generated mockup

<https://www.figma.com/make/q5Hk6pgiKKhHKBy0yFQ2Tw/Podcast-and-Statistics-App?p=f&t=HKDqDETgN1Xh9val-0&preview-route=%2Fdiscussion>